

Software Development Life Cycle (SDLC)

Week 6 – Software Planning and Development

Prepared by: Vincent Onchieku Collins

1. What Is the SDLC?

The Software Development Life Cycle (SDLC) is a structured, iterative methodology used by development teams to plan, design, build, test, deploy and maintain high-quality, cost-effective software systems. It breaks software development down into distinct, repeatable, interdependent phases, each with its own objectives and deliverables that feed into the next phase. Together, these phases form a roadmap that helps teams create software that meets stakeholder needs, project requirements and customer expectations.

There are various SDLC models, and each approaches the phases differently. In sequential models such as Waterfall, phases are completed one after another; in more iterative processes such as Agile, phases can be worked on in parallel. Regardless of model, the SDLC provides a systematic workflow that helps align software development with business goals and user requirements.

Why the SDLC Matters

- Provides a clear, organized framework for managing development phases.
- Helps in early detection of defects, reducing overall cost and time.
- Ensures high-quality software delivery that meets user expectations.
- Helps stakeholders estimate project costs and time frames, identify challenges and address risk factors early.
- Measures development progress and enhances documentation and transparency.
- Aligns software projects with organizational goals.

Sources: GeeksforGeeks – “Software Development Life Cycle (SDLC)”; *IBM – “What is the software development lifecycle (SDLC)?”*

2. The Phases of the SDLC

Practitioners broadly agree that the SDLC has seven key phases. The table below (from IBM) summarizes each phase at a glance; the detailed breakdown that follows (combining GeeksforGeeks and IBM) describes the activities, outputs and key roles involved in each phase.

Phase	Key Activities	Deliverables
1. Planning	Identify project scope, goals and requirements	Initial project plan
2. Analysis	Gather and review data on project requirements	Fully detailed requirement documentation
3. Design	Define project architecture	Software Design Document (SDD)
4. Coding	Write initial code	Functional software prototype
5. Testing	Review code and eliminate bugs	Refined, optimized software

Phase	Key Activities	Deliverables
6. Deployment	Deploy code to production environment	Software available to end users
7. Maintenance	Continual fixes and improvements	Updated and optimized code

Source: IBM – “What are the 7 phases of the SDLC?”

Phase 1: Planning & Feasibility Analysis

This stage establishes the goals and scope of the project and determines whether it is technically, financially and operationally feasible. The team brainstorms high-level project details — the problem or use case the software will solve, who will use it, and how it will interact with other systems — and solicits input from stakeholders such as business analysts, line-of-business managers, and internal and external customers. The planning phase aims to pinpoint the project's goals and equally establish what the project does not need, preventing scope bloat.

- **Activities:** Feasibility analysis, cost estimation, scheduling, resource planning, brainstorming of high-level project details.
- **Output:** Project Plan, Feasibility Report, and often an initial Software Requirement Specification (SRS) detailing functions, required resources, possible risks and timeline.
- **Key Roles:** Project Managers, Senior Engineers, Stakeholders.

Phase 2: Analysis / Requirement Specification

During this phase, detailed functional and non-functional requirements are gathered, documented and validated with stakeholders. The team moves from a high-level idea to a practical implementation plan by gathering user requirements, conducting market research and feasibility testing, evaluating prototypes and allocating resources. By the end of this phase, project managers and development teams fully understand the project's scope, its functional and technical specifications, and how to organize tasks and workflows.

- **Activities:** Requirement gathering, validation, documentation, feasibility testing, resource allocation.
- **Output:** Software Requirement Specification (SRS) / fully detailed requirement documentation.
- **Key Roles:** Business Analysts, Product Owners.

Phase 3: Design

The approved requirements are transformed into a technical blueprint for implementation. This includes outlining the software's architecture, navigation, user interfaces, database design and technology stack, and considering how the software fits into an organization's existing landscape of apps, services and dependencies.

- **High-Level Design (HLD):** Defines system architecture, technology stack, database design, and major modules.
- **Low-Level Design (LLD):** Specifies component logic, APIs, data structures, and workflows.

The design phase often includes threat-modeling exercises to identify security risks (for example, incorporating strong authentication if identity theft is a significant risk), and may use a modular or microservices architecture

— self-contained, loosely coupled components that can be developed and reused independently. The phase typically culminates in an early prototype shown to stakeholders for feedback.

- **Output:** Design Document Specification (DDS) / Software Design Document (SDD), which is passed to developers as a roadmap for coding.
- **Key Roles:** Architects, Senior Engineers.

Phase 4: Development (Coding)

Developers write the code and build the software based on the approved SDD, SRS and other guidelines from earlier phases. These documents guide the choice of programming language and help project managers divide the project into smaller, discrete coding tasks. This phase also covers building any additional systems or interfaces needed — such as web pages or APIs — for the software to function properly. Depending on the SDLC model followed, some teams perform code reviews and tests during this stage to catch bugs and vulnerabilities early.

- **Activities:** Coding, code reviews, unit testing, version control management.
- **Tools:** IDEs, version control systems, debuggers.
- **Output:** Source code, executable application / functional software prototype.
- **Key Roles:** Frontend, Backend and Full-Stack Developers.

Phase 5: Testing

Testing ensures the software meets requirements and is free of defects before release. A quality assurance team performs multiple types of testing to confirm every part of the software works as intended, meets user and business requirements, and functions correctly within the organization's broader IT environment. Testers also scrutinize the software for security vulnerabilities and document their findings; developers then use the results to fix bugs before the software is retested. Many SDLC models use continuous testing, meaning testing is not confined to a single phase but occurs throughout development.

- **Unit Testing:** Verifies individual components.
- **Integration Testing:** Ensures modules work together.
- **System Testing:** Validates the complete system.
- **User Acceptance Testing (UAT):** Confirms business requirements are met.
- **Output:** Test cases, defect reports, quality metrics, and a refined, optimized piece of software.

Phase 6: Deployment

The tested software is released to users in the production environment. The goal is not just to get the software into users' hands, but to ensure they understand how to use it and that deployment causes minimal disruption to user experience or workflows. Teams may deploy in phases — such as a beta release to a limited group — before general availability, and may create manuals, run training sessions, or offer onsite support.

- **Activities:** Production setup, deployment, smoke testing, phased/beta releases.

- **Modern Approach:** Continuous Integration and Continuous Deployment (CI/CD) pipelines for faster, more reliable releases.
- **Output:** Live application, software available to end users.
- **Key Roles:** DevOps Engineers, Release Managers.

Phase 7: Maintenance

The SDLC does not end at deployment. The maintenance phase covers the post-deployment work that keeps the software running: pushing updates and optimizations, making unanticipated changes, testing patches, addressing new use cases, and fixing bugs users find. Ongoing maintenance and support safeguard the software's long-term usability — much like maintaining a house, where parts wear down over time and must be replaced or improved. In DevOps-style models, maintenance is an ongoing activity rather than a distinct phase, since code is continuously integrated, tested and deployed.

- **Activities:** Bug fixes, performance tuning, updates, feature enhancements.
- **Output:** Patches, updates, new versions, updated and optimized code.
- **Key Roles:** Support Engineers, Developers.

Sources: GeeksforGeeks – “Stages of the Software Development Life Cycle”; IBM – phase-by-phase breakdown (Planning through Maintenance).

3. Common SDLC Models

Software development models are structured frameworks that guide the planning, execution and delivery of software projects. They define the sequence in which requirements, design, coding, testing and deployment are handled. Choosing the right model depends on factors such as how clearly requirements are defined, how likely they are to change, project complexity, and team experience.

- **Waterfall:** A linear, sequential model in which one stage is finished before the next begins. Structured and predictable, it suits well-defined, stable projects, but it is inflexible — correcting earlier phases after completion is difficult and time-consuming.
- **V-Model (Verification and Validation Model):** A variation of Waterfall in which each SDLC phase has its own accompanying testing phase. Frequent testing catches bugs early, but the model remains linear and less flexible; it suits software with stable requirements that needs frequent testing.
- **Agile:** Runs on continuous improvement and development cycles (“sprints”) with regular small, incremental releases. Well suited to projects where clients participate frequently in discussions and reviews, and responsive to changing requirements.
- **Lean:** Applies manufacturing principles to reduce waste at every step of the SDLC, prioritizing iteration, fast feedback loops and removal of bureaucratic decision-making.
- **Iterative:** An initial version or minimum viable product (MVP) is built quickly and then improved through successive versions, starting small and building outward.
- **Spiral:** Four phases — determining objectives, resource and risk analysis, development and testing, and planning the next iteration — repeat in a cycle, giving multiple opportunities for correction. Ideal for high-risk or complex projects.

- **Big Bang:** An informal, unstructured approach with little or no planning or requirement analysis. High risk, but can work for small, self-explanatory projects; relies on ad hoc tester and user feedback.
- **Rapid Application Development (RAD):** Relies on fast prototyping and user feedback rather than a long planning period, enabling quick adaptation to new needs; more structured than Big Bang and feasible for larger projects.
- **Incremental Model:** Referenced by GeeksforGeeks among the common SDLC models, alongside Waterfall, Agile, V-Model, Spiral and RAD.

Sources: GeeksforGeeks – “Software Development Life Cycle Models” (Common Models list); IBM – “Common software development lifecycle models.”

4. Agile, Scrum and Kanban

Agile is one of the most popular SDLC models because it emphasizes collaboration, continuous delivery and customer feedback. It breaks large projects into time-boxed “sprints” — smaller tasks with discrete goals meant to be completed in short time frames — keeping teams feature-driven and able to quickly identify and respond to changing requirements.

- **Scrum:** An agile project-management framework in which development teams are broken into smaller units led by a “scrum leader,” who reports to the product owner. Each team takes full ownership of its assigned task in each sprint, enabling adaptability without waiting on other stakeholders.
- **Kanban:** A continuous-workflow agile framework (as opposed to scrum's time-boxed sprints). Tasks are shown visually on a kanban board so team members can see what remains and move immediately to the next step as tasks are completed.

Source: IBM – “SDLC, agile and scrum.”

5. DevOps, DevSecOps and Security in the SDLC

DevOps and the SDLC

DevOps is a software development methodology that combines and automates the work of development and IT operations teams. Its core principles are collaboration, automation, and continuous integration and continuous delivery (CI/CD). Rather than treating SDLC phases as discrete steps, DevOps makes planning, coding, testing, deploying, maintaining and monitoring continuous throughout the product's lifecycle, resulting in a continuous delivery pipeline with frequent updates. DevOps is broader than a single model — it is also a cultural and organizational shift toward shared responsibility and collaboration.

DevSecOps and Embedding Security

DevSecOps (“secure DevOps”) integrates automated security testing and practices into the DevOps model. Where traditional development treats security testing as its own phase, DevSecOps builds security into every stage of the SDLC — from design through deployment — via practices such as code reviews and penetration testing. This reduces risk, improves software resilience, and enables safer, more cost-effective applications.

- Vulnerabilities are identified and fixed early in the development process.
- Automated security checks are integrated into build and CI/CD pipelines.

- Security becomes a shared responsibility across development, testing and operations teams.

Sources: IBM – “DevOps and the SDLC”, “DevSecOps and the software development lifecycle”; GeeksforGeeks – “Embedding Security into the SDLC.”

6. AI in the SDLC

According to an IEEE survey cited by IBM, 35% of organizations across industries are using artificial intelligence to aid or accelerate software development. Generative AI tools are increasingly integrated into all stages of the SDLC: creating software prototypes and reusable code snippets, helping developers refine code, flagging and explaining errors, and analyzing test data to identify trends and patterns in performance and failures. In “vibe coding,” developers describe desired functionality in plain text and let an AI tool generate the initial code, which is then refined further.

AI adoption is not without risk. AI tools can make mistakes or produce unoptimized code, and without careful review, they can introduce costly bugs that surface only much later in the lifecycle. Balancing quality and speed is also a challenge: while AI can speed up coding, ensuring the quality of AI-generated output requires human oversight and validation, which can offset the time savings.

Source: IBM – “AI in the SDLC.”

7. Benefits of the SDLC

The SDLC gives development teams a standardized, repeatable structure that makes it easier to consistently deliver high-quality software.

- **Better software:** A structured map for completing complex projects within schedule and budget, with testing and quality assurance built in.
- **Improved productivity:** Streamlines projects and eliminates guesswork through clear documentation between phases.
- **Minimized risk:** Enables continuous risk assessment so problems are caught and mitigated before they grow.
- **Improved project transparency:** Standardized documentation and communication keep stakeholders informed of progress and responsibilities.
- **Greater collaboration:** Encourages alignment on goals and pain points, and in some models, small, highly collaborative teams.
- **More efficient resource management:** Upfront cost and resource estimation helps decrease bloat and keep projects on budget.
- **Improved customer satisfaction:** Rigorous planning, building and testing reduce feature bloat, improve usability, and result in fewer bugs at launch.

Source: IBM – “SDLC benefits.”

8. Common Challenges and Mistakes

Common Challenges (IBM)

- **Staying within project scope:** Scope creep — requirements expanding beyond the initial plan — can blow through budgets and derail development, as can chasing perfection instead of a functional, iterable release.
- **Poorly defined requirements:** Failing to thoroughly analyze requirements up front can cause wasted work cycles, delayed releases and higher costs.
- **Determining the optimal level of testing:** Testing can account for nearly a third of system development costs; too little testing risks bugs reaching users, while excessive testing can needlessly delay release.
- **Securing software updates and maintenance:** Supply-chain attacks — such as the 2020 SolarWinds incident, where hijacked update processes spread malware to government agencies — show why application-security measures must extend into post-deployment maintenance.

Common Mistakes Teams Make (GeeksforGeeks)

- **Treating documentation as the goal:** Producing SRS and design documents without validating assumptions with real users.
- **Late or inadequate testing:** Testing rushed or reduced under time pressure, increasing post-release defects.
- **Ignoring non-functional requirements:** Performance, security and scalability considered too late.
- **Poor communication between teams:** Gaps between business, development and QA lead to misaligned outcomes.
- **Skipping feedback loops:** Limited iteration and user feedback let issues surface only after deployment.
- **Overengineering early:** Designing overly complex solutions before validating core functionality.

Avoiding these pitfalls requires continuous collaboration, early validation, and a mindset that values outcomes over process for its own sake.

Sources: IBM — “Common challenges in the software development lifecycle”; GeeksforGeeks — “Common SDLC Mistakes Teams Make.”

9. Real-Life Example: Banking Application Development

GeeksforGeeks illustrates the SDLC using the example of building a banking application:

- **Planning & Analysis:** Identify banking features such as account management, fund transfers, and security requirements.
- **Design:** Create UI designs, system architecture, databases, and choose the technology stack.
- **Development:** Implement frontend interfaces, backend services, and APIs.
- **Testing:** Conduct functional, performance and security testing.
- **Deployment & Maintenance:** Release the application and continuously monitor it, fix defects, and add enhancements.

Source: GeeksforGeeks — “Real Life Example of SDLC.”

10. Why Projects Can Still Fail Despite Following the SDLC

Following the SDLC does not automatically guarantee success. Many projects fail because teams treat it as a checklist rather than a decision-making framework — the SDLC provides structure, but success depends on how thoughtfully it is applied.

- **Poor requirement clarity:** Requirements are documented but not deeply understood, leading to incorrect assumptions and rework.
- **Weak stakeholder involvement:** Limited feedback from users or business teams results in solutions that don't solve real problems.
- **Rigid execution:** Teams follow the process mechanically and resist adapting to changing business or technical realities.
- **Underestimated complexity:** Risks related to scalability, integration or performance are identified too late.
- **Lack of ownership:** Roles exist on paper, but accountability for outcomes is unclear.

Source: GeeksforGeeks – “Reasons for Project Failure Despite Following SDLC.”

Compiled from: GeeksforGeeks – “Software Development Life Cycle (SDLC)” and IBM – “What is the software development lifecycle (SDLC)?”. No content beyond these two sources has been added.